



Guia do Aluno — Diagrama de Classes UML

UC: Programação Back-End | **Abertura do Bloco 2 — Aula de 21/08/2026**

Projeto: API de Gestão da Livraria (SA1)

O que muda hoje

Desde 29/07 vocês escrevem código primeiro. **Hoje é o dia em que isso se inverte.** UML é uma linguagem de **desenho**, não de código — serve para pensar a estrutura de um sistema **antes** de escrever qualquer linha em JavaScript. Vocês vão inclusive desenhar classes que ainda não existem em código nenhum: `Pedido` e `ItemPedido`.

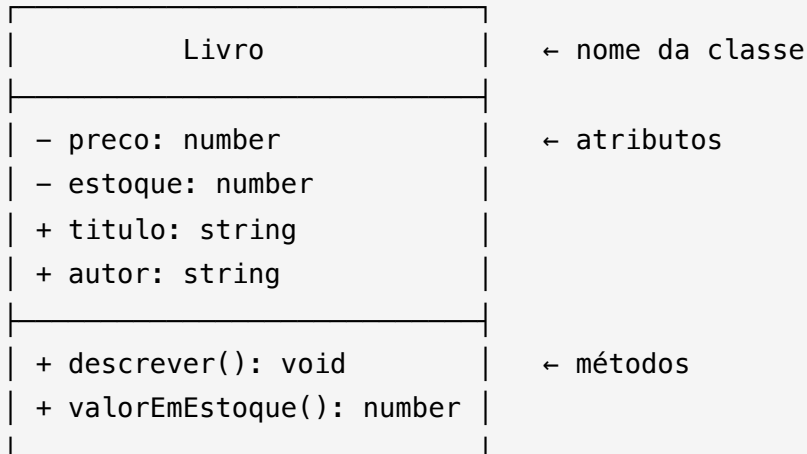
Ferramenta de hoje: app.diagrams.net ([Draw.io](https://draw.io)). Nenhum código, nenhum terminal.

O que você vai conseguir fazer ao final

- ☐ Desenhar a caixa de uma classe, com atributos e métodos na notação UML
 - ☐ Diferenciar os 4 tipos de relacionamento entre classes: associação, agregação, composição e herança
 - ☐ Explicar por que a diferença entre agregação e composição importa
 - ☐ Usar multiplicidade para dizer "quantos de cada lado"
 - ☐ Começar, em grupo, o diagrama de classes da Livraria no [Draw.io](https://draw.io)
-

1. A caixa de classe

Toda classe em UML se desenha assim, sempre com **3 compartimentos**, nesta ordem:



A notação é a mesma coisa que vocês já escrevem em JavaScript, só desenhada:

Símbolo UML	Igual a, no código
- (menos)	atributo com # na frente (privado)
+ (mais)	atributo ou método sem # (público)
nome: tipo	um parâmetro do constructor , com o tipo do dado ao lado
metodo(): tipo	um método, com o que ele devolve entre parênteses

💡 : **void** significa "este método não devolve nada com `return` , só executa uma ação".
`descrever()` só imprime, então é : **void** . Já `valorEmEstoque()` devolve um número com `return` , então é : **number** .

2. Os 4 tipos de relacionamento

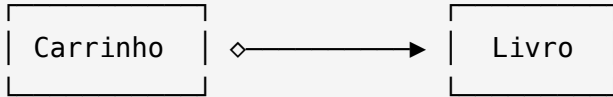
Do mais fraco para o mais forte:

🔗 Associação — "conhece" ou "usa"



Linha simples com seta aberta. O `Funcionario` **sabe** que existe um `Cliente` e interage com ele (atende), mas não é "dono" dele — o cliente existe de forma completamente independente.

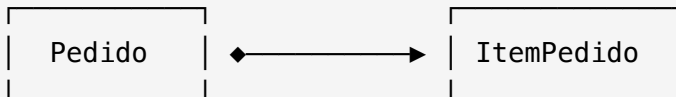
🕒 Agregação — "tem um" fraco (losango VAZIO)



Vocês já usaram essa relação em 12/08, com a classe `Carrinho`. Hoje ela ganha nome próprio: **agregação**.

O teste: "se eu apagar o `Carrinho`, o `Livro` deixa de existir na livraria?" Não — ele continua na prateleira, disponível para outro carrinho. **A parte sobrevive sem o todo.** É por isso que o losango é **vazio**: a ligação é "fraca".

■ Composição — "tem um" forte (losango PREENCHIDO)



`Pedido` e `ItemPedido` são classes **novas hoje** — ainda não existem em nenhum código de vocês.

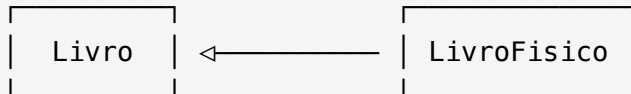
O teste: "um `ItemPedido` faz sentido sozinho, sem o `Pedido` dele?" Não — se o pedido for apagado, os itens dele não fazem mais sentido separados. **A parte NÃO sobrevive sem o todo.** Por isso o losango é **preenchido**: a ligação é "forte".

★ O contraste que importa

AGREGAÇÃO (losango vazio) → a parte SOBREVIVE sem o todo → `Carrinho` / `Livro`
COMPOSIÇÃO (losango cheio) → a parte NÃO SOBREVIVE sem o todo → `Pedido` / `ItemPedido`

🔍 **Pergunta rápida:** e a relação `Livro` / `Categoria`, que vocês fizeram em 12/08 — é agregação ou composição? Pense no teste: se um livro for apagado, a categoria "Tecnologia" deixa de existir? Não — outros livros continuam usando ela. Então é **agregação**, igual a `Carrinho` / `Livro`.

▲ Herança — "é um" (triângulo VAZIO, aponta para a classe-mãe)



Essa vocês já dominam desde 07/08 (`extends`). A única coisa nova é o símbolo: **triângulo vazio**, e a seta sempre aponta **para a classe-mãe**, nunca para a filha.

Quadro-resumo

Relacionamento	Símbolo na ponta	Pergunta-teste	Exemplo da Livraria
Associação	seta simples aberta	"Ele conhece/usa o outro?"	Funcionario → Cliente
Agregação	losango vazio	"A parte sobrevive sem o todo?" (sim)	Carrinho ◇→ Livro
Composição	losango cheio	"A parte sobrevive sem o todo?" (não)	Pedido ◆→ ItemPedido
Herança	triângulo vazio	"É um tipo de...?"	LivroFisico ◁— Livro

3. Multiplicidade — "quantos de cada lado"

Todo relacionamento pode dizer **quantos objetos** participam de cada lado:

Carrinho "1" ◇————→ "0..*" Livro

Lê-se: "um carrinho tem de zero a muitos livros."

Notação	Significa
1	exatamente um
0..1	zero ou um
0..* (ou só *)	zero ou muitos
1..*	um ou muitos (pelo menos um)

Exemplo aplicado: um Pedido precisa ter **pelo menos um** item (pedido vazio não faz sentido) → 1..*. Já cada ItemPedido pertence a **exatamente um** pedido → 1.

Pedido "1"  "1..*" ItemPedido

4. Usando o Draw.io

1. Acesse app.diagrams.net. Escolha **Device** (não precisa criar conta).
2. Na barra de busca de formas (lado esquerdo), digite "**class**" e arraste a forma de classe UML pronta — ela já vem com os 3 compartimentos.
3. Edite o texto de cada compartimento com duplo clique.
4. Para conectar duas classes: passe o mouse sobre a borda de uma caixa até aparecer uma setinha azul, clique e arraste até a outra caixa.
5. **Para trocar o tipo de relacionamento:** clique na linha desenhada. No painel do lado direito, procure as opções de estilo da linha e da ponta (seta). Escolha:
 - Seta aberta simples → associação
 - Losango vazio → agregação
 - Losango cheio → composição
 - Triângulo vazio → herança
6. Para escrever a multiplicidade, dê duplo clique **perto da ponta da linha** (não em cima da linha toda) e digite o número.
7. **Salvar:** Ctrl + S salva no navegador. Para exportar como imagem: menu **File** → **Export as** → **PNG**.

5. A atividade de hoje

Em grupo, vocês vão montar o **rascunho inicial** do diagrama de classes da Livraria, incluindo as classes que já existem em código e duas novas (`Pedido` e `ItemPedido`). Sigam o arquivo `03-atividade-diagrama-uml-grupo.md` .

Não precisa terminar hoje — é rascunho. A gente retoma e refina em 21/08.

Checklist

- ☐ Sei desenhar uma caixa de classe com os 3 compartimentos corretos
- ☐ Sei diferenciar `-` (privado) de `+` (público) na notação
- ☐ Consigo explicar a diferença entre agregação e composição com o teste "a parte sobrevive?"
- ☐ Sei qual símbolo usa cada um dos 4 relacionamentos
- ☐ Sei escrever multiplicidade em uma relação
- ☐ Consegui usar o [Draw.io](https://draw.io) para desenhar e conectar caixas
- ☐ O grupo salvou/exportou o progresso do diagrama antes de sair

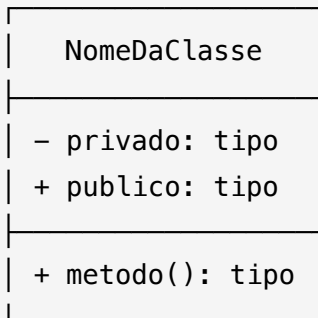
Erros comuns

Sintoma	O que aconteceu	Como resolver
Losango vazio e cheio parecem iguais no desenho	Cliques errados no painel de estilo do Draw.io	Confira no painel direito qual ponta está selecionada (start/end)
Seta de herança apontando para a classe errada	Direção invertida	O triângulo sempre aponta para a classe-mãe
Confundi agregação com composição	Não aplicou o teste da pergunta	Pergunte sempre: "se eu apagar o todo, a parte desaparece?"
Método sem <code>: tipo</code> no diagrama	Esqueceu de indicar o retorno	Métodos que não usam <code>return</code> levam <code>: void</code>

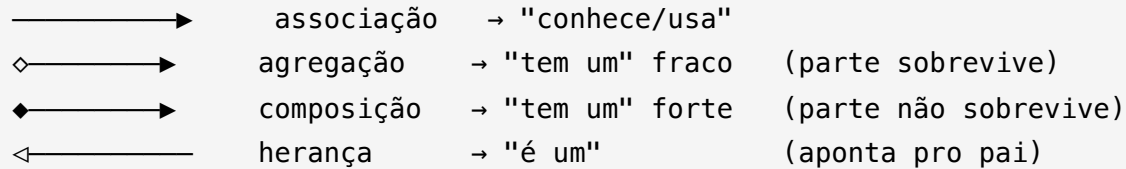
Sintoma	O que aconteceu	Como resolver
Multiplicidade <code>0..*</code> num caso que exige pelo menos 1	Não pensou na regra de negócio	Releia o exemplo do <code>Pedido</code> , que exige <code>1..*</code>

Cola rápida

NOTAÇÃO DA CLASSE



RELACIONAMENTOS (do mais fraco ao mais forte)



MULTIPLICIDADE

1	exatamente um
0..1	zero ou um
0..*	zero ou muitos
1..*	um ou muitos (pelo menos um)